

Point Supervised Remote Sensing Segmentation

By Stephen Austine Wanyama

What I worked on

This report covers three things that were required from the assessment which include, implementing the Partial Cross-Entropy (pCE) loss, building and training a small segmentation model on a remote sensing dataset using simulated point labels, and running two experiments of which one was looking at how much annotation density matters, and the other one was comparing the pCE loss against standard CE and a focal weighted version.

For the dataset, I used the Satellite Image Classification dataset from Kaggle which has four classes. These include, cloudy, desert, green_area, and water, that map reasonably well onto a remote sensing segmentation setting.

Dataset

The dataset has roughly 5,600 images across four classes: **cloudy**, **desert**, **green_area**, and **water**. The images are not all the same size since cloudy and desert are 256×256 , while green_area and water are 64×64 , so I went on and resized everything to **64×64** before passing them to the model.

There are no pixel-level masks in this dataset, only image-level class labels. For the point supervision setup I use the image-level class as the label for each pixel and then keep only a randomly selected fraction of those pixels as labelled points. The rest are marked as unlabeled (target = -1) and the loss ignores them. Figure 1 shows what the simulated point annotations look like at 10% density.

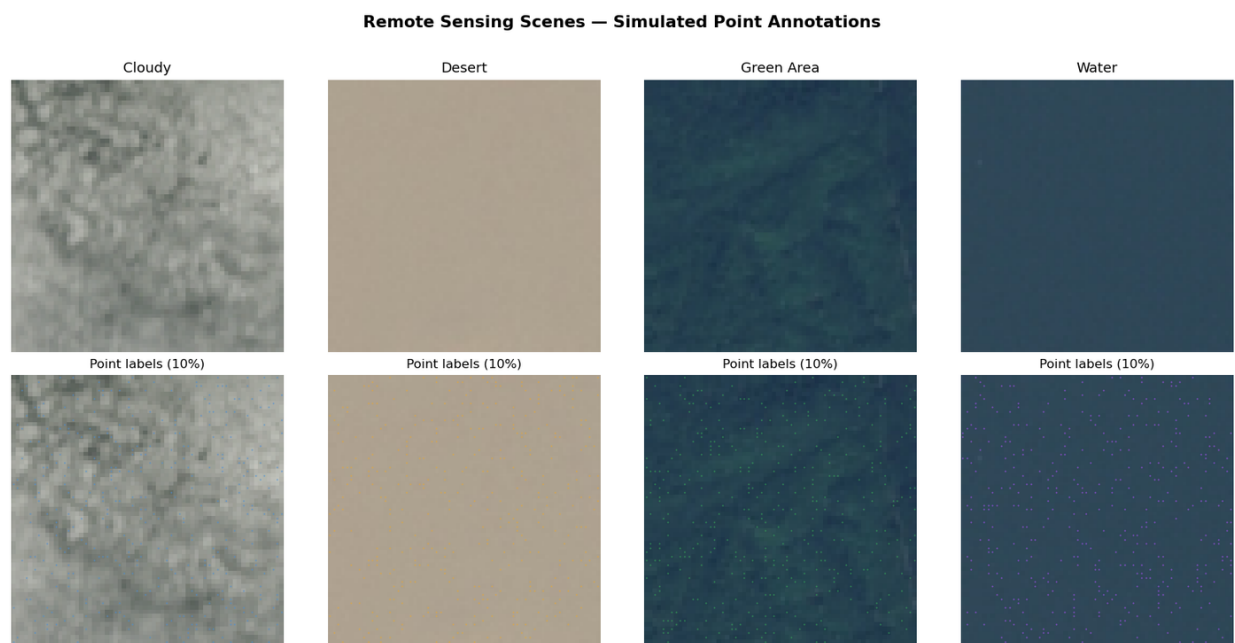


Figure 1. Simulated point annotations at 10% density across the four classes. Top row: original images, Bottom row: the sampled pixel positions shown as coloured dots.

Partial Cross-Entropy Loss

With sparse point labels, most pixels don't have a target, the loss therefore only uses the pixels for which a point label is available. This is the partial CE formulation described in the assessment, which follows the approach used in point-supervised segmentation work. The formula from the assessment document:

$$\text{pCE} = \sum \text{CE}(\text{pred_i}, \text{gt_i}) \cdot \text{mask_i} / \sum \text{mask_i}$$

I use ordinary cross-entropy when $\gamma=0$. The important part is that the average is taken over the labelled pixels rather than over the whole image, without that, the gradient gets diluted by all the unlabeled positions and training becomes noisy.

In code I flatten both the logit and target tensors, use the boolean point mask to pull out only the labelled positions, then call `F.cross_entropy(reduction="none")` to get per-pixel values before dividing by `mask.sum()`. If $\text{focal_gamma} > 0$, I also compute the focal weight $(1 - p_t)^\gamma$ per pixel and multiply it in before averaging.

Segmentation Model

The model (PointSegNet) uses a pretrained ResNet-18 as the encoder, loaded with ImageNet weights via torchvision. I stop the encoder at layer 3 so that more spatial information is kept, dropping layer 4 means the feature map stays at a higher resolution which matters at 64×64 input where each stride cuts into an already small grid. The decoder is two 3×3 conv blocks with BatchNorm and ReLU, then bilinear upsampling back to the input size. The output is a 4-channel logit map.

$$3 \times 64 \times 64 \rightarrow \text{ResNet-18 encoder (layers 1-3)} \rightarrow \text{decoder} \rightarrow 4 \times 64 \times 64$$

~3.15M parameters. Pretrained ImageNet weights on the encoder. Trained with PyTorch using Adam ($\text{lr}=1\text{e-}3$, $\text{wd}=1\text{e-}4$). Cosine annealing over 20 epochs. Batch size 32. Augmentation: horizontal flips, vertical flips, colour jitter.

Main Training Run

Using 10% point density and a pretrained ResNet-18 encoder.

The model reached **99.6% image-level validation accuracy** by epoch 20. Train and val loss both decreased together throughout and val loss was consistently lower than train loss, which makes sense because augmentation and BatchNorm make training harder than evaluation, but there was no sign of overfitting.

The high accuracy is not surprising given the dataset, four visually distinct classes where a pretrained ResNet-18 already has most of the features it needs before any task-specific training. The experiments below test the more interesting edge cases.

Experiments

Experiment 1: Point Density

Question

How much does the model change when fewer or more pixels are labelled?

Hypothesis

More labelled points should give the model more supervision so performance should generally improve as the point rate goes up. I also wanted to see whether the model can still learn anything useful with very sparse labels, given that the pretrained encoder is already doing a lot of the work.

Setup

point_rate was tested at 0.01, 0.05, 0.10, and 0.20, everything else such as the encoder weights, epochs, optimizer and seed, stayed the same.

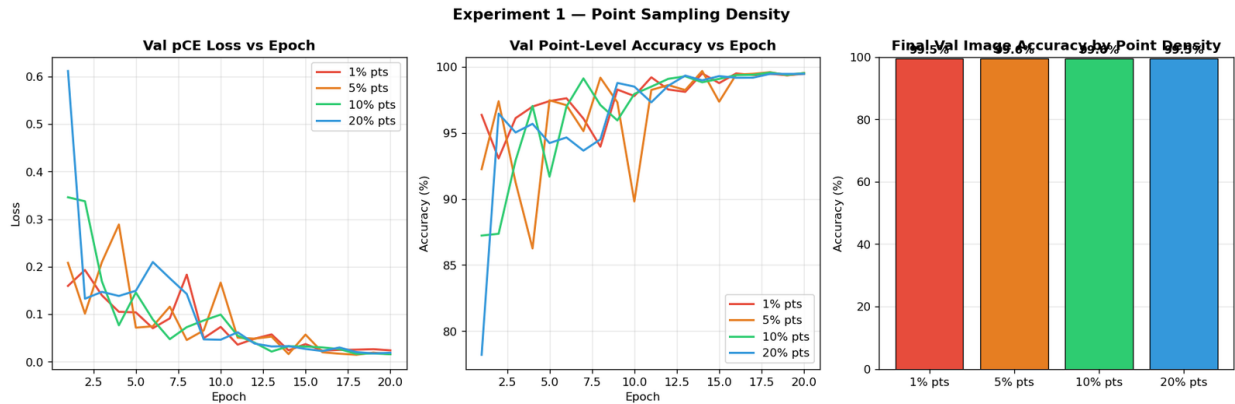


Figure 2. Experiment 1 results. Left: val pCE loss over 20 epochs, Centre: val point-level accuracy, Right: final image-level accuracy for each point density setting.

Results

Point Rate	Final Val Loss	Pt-Level Acc (%)	Img-Level Acc (%)
1%	~0.02	~99%	99.5%
5%	~0.02	~99%	99.6%
10%	~0.02	~99%	99.0%
20%	~0.02	~99%	99.5%

All four settings ended up above 99% image accuracy, so the final bar chart looks flat. The loss curves are more informative since the 1% and 5% runs show more variance in the early epochs because fewer pixels are contributing gradient per batch, but note that everything stabilises around epoch 10.

The model is still able to learn at 1% point density which I think is mostly down to the pretrained encoder. The biggest change between settings is in early epoch stability rather than final accuracy. For this dataset the ceiling gets hit early, but on a harder problem with less visually distinct classes or a weaker backbone the gap would likely be much larger.

Experiment 2: Comparing the Loss Functions

Question

Does the way we handle the unlabeled pixels make a noticeable difference?

Hypothesis

Standard CE with `ignore_index=-1` and pCE with `gamma=0` should be very similar because both use only the labelled pixels and the difference is just in how the normalization is expressed. Adding focal weighting (`gamma=2`) may help the model spend more of its loss budget on harder examples, which could be useful when labeled pixels per batch are few.

Setup

Point rate fixed at 5%, three versions trained with the same configuration, only the loss function changes:

- **Standard CE** with `ignore_index=-1`
- **Partial CE** ($\gamma = 0$) explicit normalization over labelled pixels
- **Partial CE + Focal** ($\gamma=2$) focal weight applied before averaging



Figure 3. Experiment 2 results. Left: training loss, Centre: val image accuracy over epochs' Right: final val image accuracy per loss variant.

Results

Loss Function	Final Train Loss	Final Img Acc (%)
Standard CE (<code>ignore_index=-1</code>)	~0.02	99.6%
Partial CE ($\gamma=0$)	~0.02	99.6%
Partial CE + Focal ($\gamma=2$)	~0.01	99.7%

Standard CE and pCE with `gamma=0` came out almost identical, which is what I expected as they use the same labelled pixels and the only difference is how the denominator is written. The focal version ended up with a slightly lower training loss and a marginal accuracy improvement at 99.7%. Looking at the val accuracy curves, the focal model shows less variance mid-training with fewer

dips, which makes sense as down-weighting the easy examples puts more of the gradient on the harder ones, and that seems to help when only 5% of pixels are labelled per batch.

The differences here are fairly small because the classes separate easily and the result does confirm that the pCE implementation is working correctly as it gives the same answer as PyTorch's native ignore mechanism while making the normalization explicit and making it easy to add things like focal weighting on top.

Qualitative Results

Here I take one validation image from each class and compare the input with the segmentation predicted by the main model.

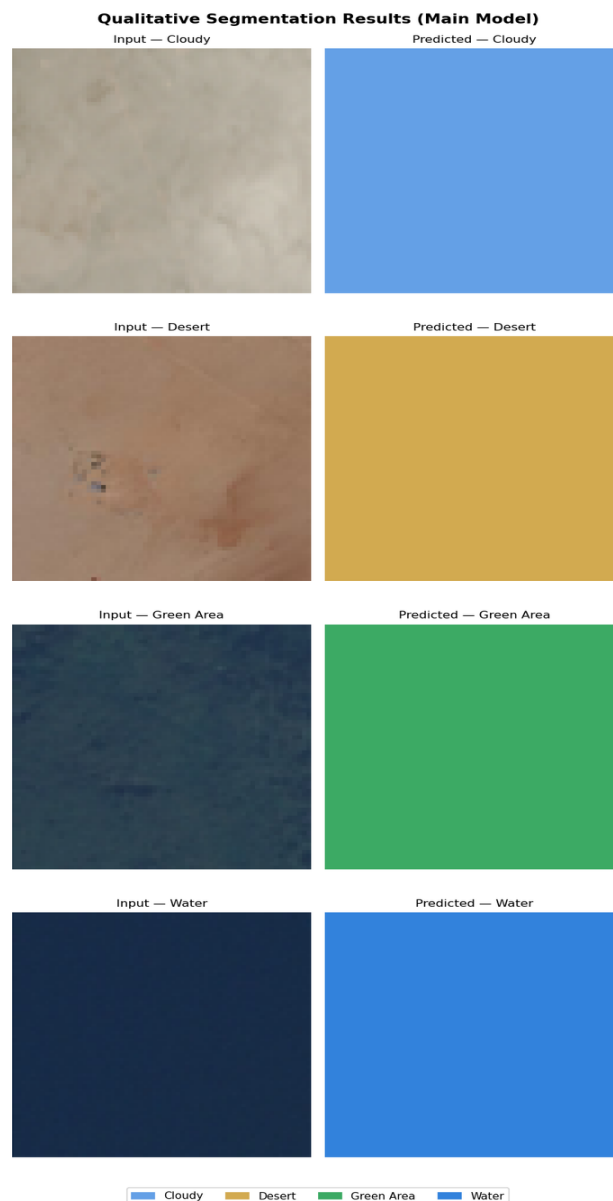


Figure 4. Qualitative segmentation predictions from the main model. Left column: input images, Right column: predicted class maps.

The model gets the correct class for all four validation images. The predicted maps are spatially uniform as every pixel gets the same scene-level class, of which is expected given that the supervision is only point labels carrying the scene class. And therefore the model learns to classify the whole scene rather than produce spatially varying boundaries.

To get predictions with spatial structure, the training would need either actual pixel-level masks at some positions or a consistency loss on the unlabeled regions to encourage smoothness. The original Landvisor pipeline handles this with semi-supervised learning on the unlabeled pixels which would be the natural next step from here.

Summary

The pCE loss is a fairly simple change to the standard segmentation setup as the key part is only computing the average over the labelled pixels rather than the whole image, which stops the gradient from being diluted by all the unlabeled positions. Combined with a pretrained ResNet-18 encoder and some basic augmentation, the model can still learn from as few as 1% of pixels labeled which I think is mostly down to the encoder doing most of the representation work upfront.

The two experiments show that for this particular dataset, annotation density above 1% gives diminishing returns on final accuracy, and the specific loss formulation makes very little difference at the image level when the classes are this visually distinct. A harder benchmark with finer class boundaries and more spatial variation within scenes would probably show a bigger effect from both of these factors for which is the kind of setting the full pipeline described in the assessment is designed for.

References

Reda, M. *Satellite Image Classification*. Kaggle, 2021.

<https://www.kaggle.com/datasets/mahmoudreda55/satellite-image-classification>

Maintainers, T. and Contributors, T. *TorchVision: PyTorch's Computer Vision Library*. GitHub, 2016. <https://github.com/pytorch/vision>